STAFF REVIEW · H446 025 source models

Separate from the eight-page pupil workbook. This copy is in a public repository and is NOT access controlled. Check technical correctness before use.

Printed question mapping: Booklet Q1 = original online Q1; Booklet Q2 = original online Q2; Booklet Q3 = original online Q3

Do Now — source answers

1. 0 — C# arrays are zero indexed.

2. Linear search checks sequentially; binary search repeatedly halves the remaining sorted range.

3. 18 — the fourth item has index 3.

4. Because comparisons rely on sorted order to discard one half safely.

I Do — source worked steps

1. Declare an integer array — CODE:
int[] numbers = {3, 7, 12, 18, 24, 31, 45, 60}; — int = integer; [] = an array; numbers = its name; { } = the values; ; = end of the statement. The first index is 0.

2. Choose a target — CODE:
int target = 31; — int declares an integer variable. The value 31 is what we want to find. This is not its index.

3. Declare a function that returns a position — CODE:
static int BinarySearch(int[] data, int target)
{ — The return type int means the function gives back a whole number: the position of the target or -1. int[] data receives the sorted array; int target receives the number to find.

4. Set the search bounds — CODE:
int low = 0;
int high = data.Length - 1; — low is the first valid index (0); high is the last valid index (7). Length is 8, but index 8 does not exist.

5. Repeat and find the middle — CODE:
while (low <= high)
{
int mid = low + (high - low) / 2; — while repeats while a candidate remains; <= also checks when low and high are equal. Integer division gives a whole-number index.

6. Compare and move the bounds — CODE:
if (data[mid] == target)
return mid;
else if (data[mid] < target)
low = mid + 1;
else
high = mid - 1; — == checks equality; < means move right; otherwise move left. Each unsuccessful step removes half of the remaining candidates.

7. Finish the method — CODE:
}
return -1;
} — } closes the loop and the method. Returning -1 means the value was not found.

8. Call the method and print the result — CODE:
int position = BinarySearch(numbers, target);
Console.WriteLine(position); // 5 — Call the function from Main after declaring the array and target. Its returned integer is stored in position and printed. Here 31 is at index 5; if the target is absent, position contains -1.

We Do — guided CHECK

1. Answer and explanation
target = 13. Return -1 if 13 does not appear.

2. Answer and explanation
low = 0, high = 7.

3. Answer and explanation
mid = 3, data[3] = 18.

4. Answer and explanation
Set high = mid - 1 = 2. Keep indexes 0–2.

5. Answer and explanation
mid = 1; data[1] = 7.

6. Answer and explanation
Set low = mid + 1 = 2.

7. Answer and explanation
mid = 2; data[2] = 12; 12 is not 13.

8. Answer and explanation
No. The loop ends because 3 > 2. Return -1.

9. Answer and explanation
CODE:
int high = data.Length - 1;
while (low <= high)
{
int mid = low + (high - low) / 2;
if (data[mid] < target)
{
low = mid + 1;
}
}
If the midpoint value is smaller than the target, move the lower bound to the right.

10. Answer and explanation
The smaller midpoint means search right: set low = mid + 1, not high = mid - 1.

11. Answer and explanation
B: low=4, high=6. 12 is smaller than 21, so search the right-hand section.

You Do — source mark schemes

1. [No safely extracted source model: staff must check]

2. [No safely extracted source model: staff must check]

3. [No safely extracted source model: staff must check]

Exit Ticket — source answer

1. [No safely extracted source model: staff must check]